OOAD - Mini Project - UML Diagrams Submission

Group Number: J7

Name:

Shriya Asija,

Sohamdeep Mandal,

Srihari Bharadwaj,

Sumeet Tummaramatti

Section: J

Figure 1 -> Class Diagram with SOLID and GRASP principles

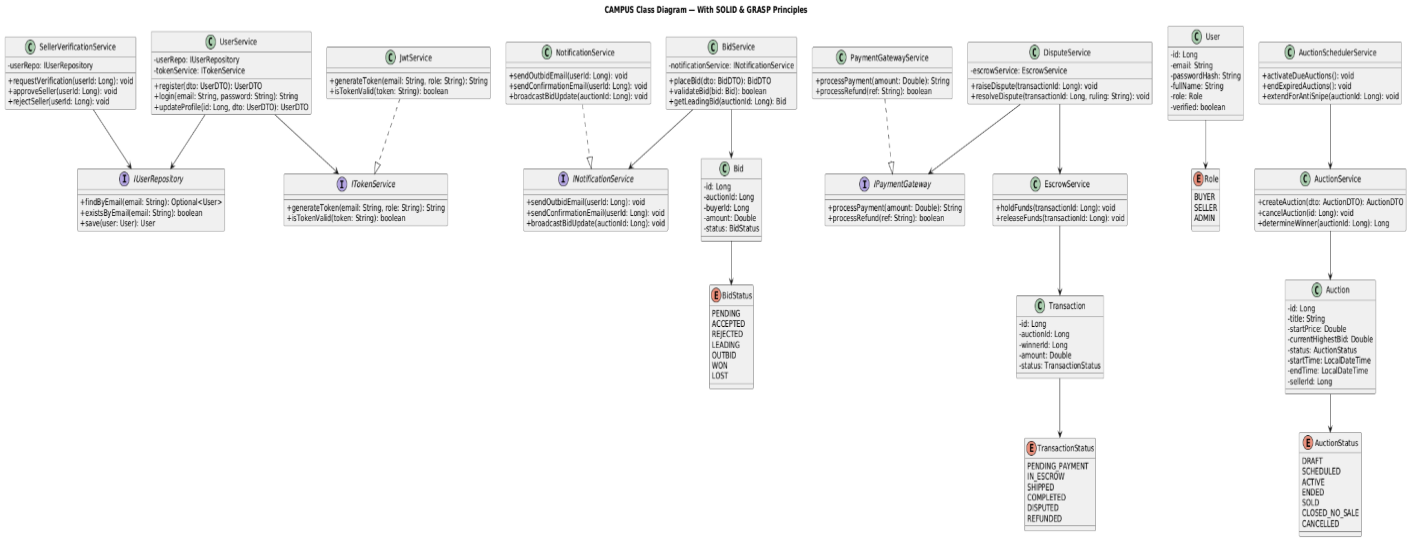
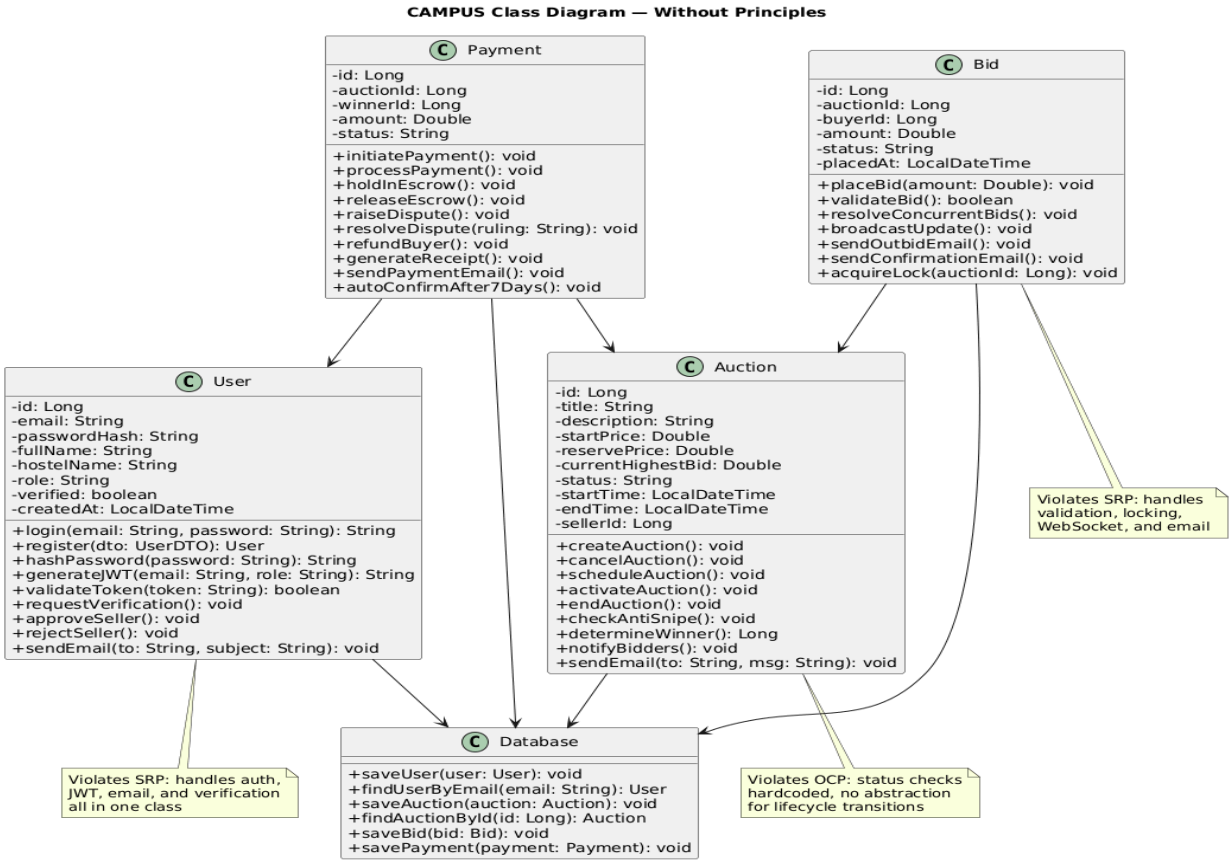


Figure 2 -> Class Diagram violating SOLID and GRASP principles



Here is our breakdown of exactly why the figure 1 is vastly superior, mapped directly to SOLID and GRASP principles:

1. The End of "God Classes" (Single Responsibility Principle & High Cohesion)

- The Flaw in Figure 2: The User class is a classic "God Class." It handles its own data (email), business logic (login), cryptography (hashPassword), security (generateJWT), and infrastructure (sendEmail). This gives the class five different reasons to change, violating SRP.

- The Fix in Figure 1: We see these concerns separated perfectly. A user is purely a domain entity that holds state. UserService handles the business workflow, JwtService handles security, and NotificationService handles infrastructure. This achieves High Cohesion (GRASP), meaning each class does exactly one thing and does it well.

2. Eliminating Tight Coupling (Dependency Inversion Principle & Low Coupling)

- The Flaw in Figure 2: Every single domain class (User, Auction, Bid, Payment) is directly tied to a concrete Database class. If we ever want to migrate from SQL to NoSQL, or change how emails are sent, we have to rewrite the core business logic.

- The Fix in figure 1: Interfaces like IUserRepository, ITokenService, and IPaymentGateway are introduced. Our business services now depend on abstractions, not concretions (DIP). This keeps the core logic completely insulated from infrastructure changes, achieving Low Coupling (GRASP).

3. Protecting Domain Entities (Indirection & Pure Fabrication)

- The Flaw in Figure 2: The Bid class assumes infrastructure roles such as acquireLock() and sendOutbidEmail(). Domain objects should never know about databases, threads, or email servers.

- The Fix in Figure 1: We use fabricated services (Pure Fabrication in GRASP) like BidService and AuctionSchedulerService to act as intermediaries. The domain objects (Bid, Auction) remain pure, while the services handle the heavy lifting of coordinating between the database, the entity, and the notification layers (Indirection).

4. Preventing Primitive Obsession (Type Safety & Open/Closed Principle)

- The Flaw in Figure 2: Using a string for status and role (e.g., "PENDING", "BUYER") is a severe anti-pattern known as "Primitive Obsession." It makes the code incredibly prone to typos and requires constant if/else checks across the application, violating the Open/Closed Principle (OCP).

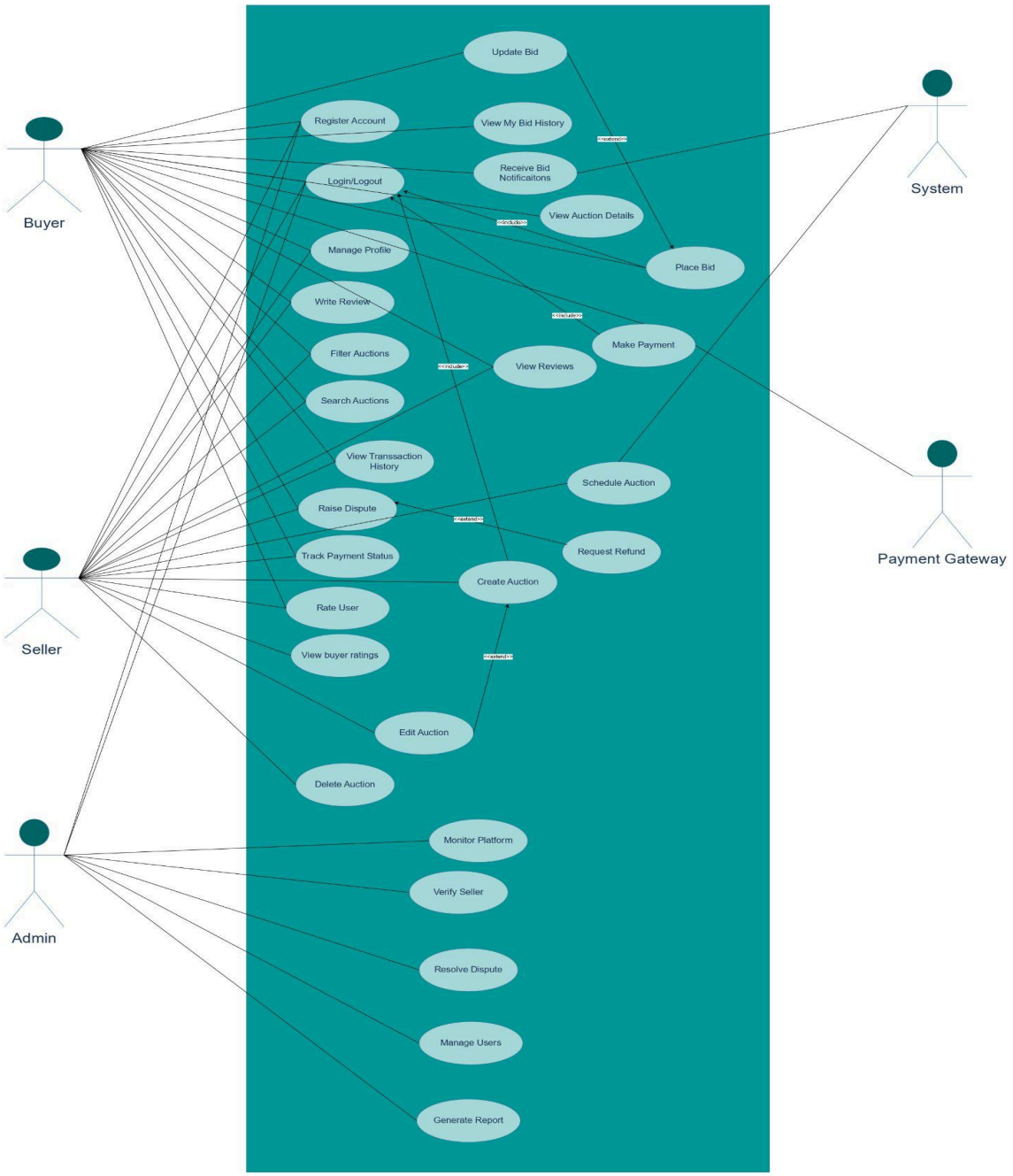
- The Fix in figure 1: Strictly defined Enums (Role, AuctionStatus, BidStatus) are utilised. This makes the code compiler-safe, self-documenting, and much easier to extend if we need to add a new state like CLOSED_NO_SALE without breaking existing logic.

5. Massive Improvements in Testability

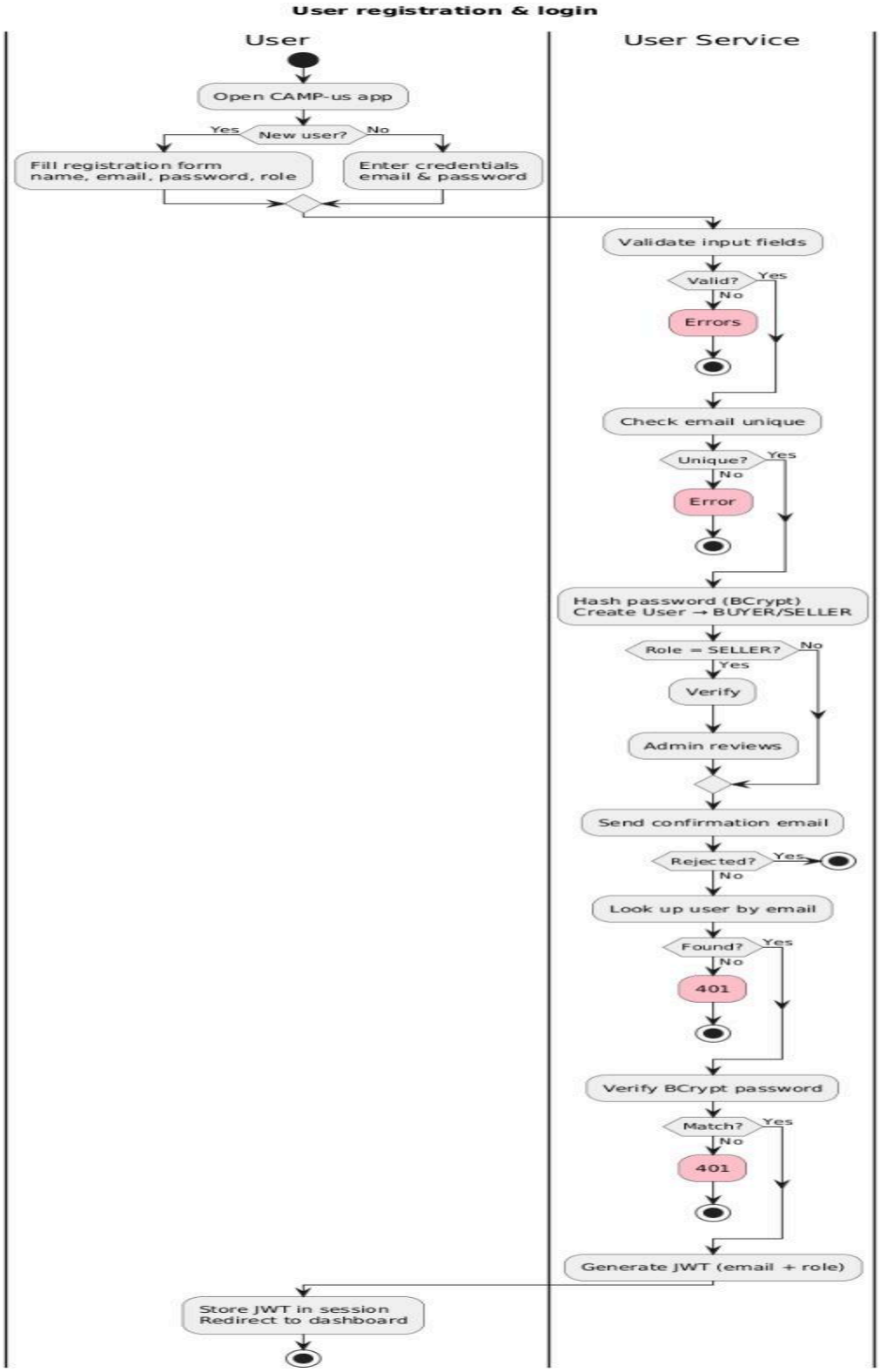
- The Flaw in Figure 2: It is physically impossible to write a simple unit test for Bid.placeBid() because doing so will automatically try to hit a real Database and fire off a real sendEmail(). We would be forced to write slow, brittle integration tests.

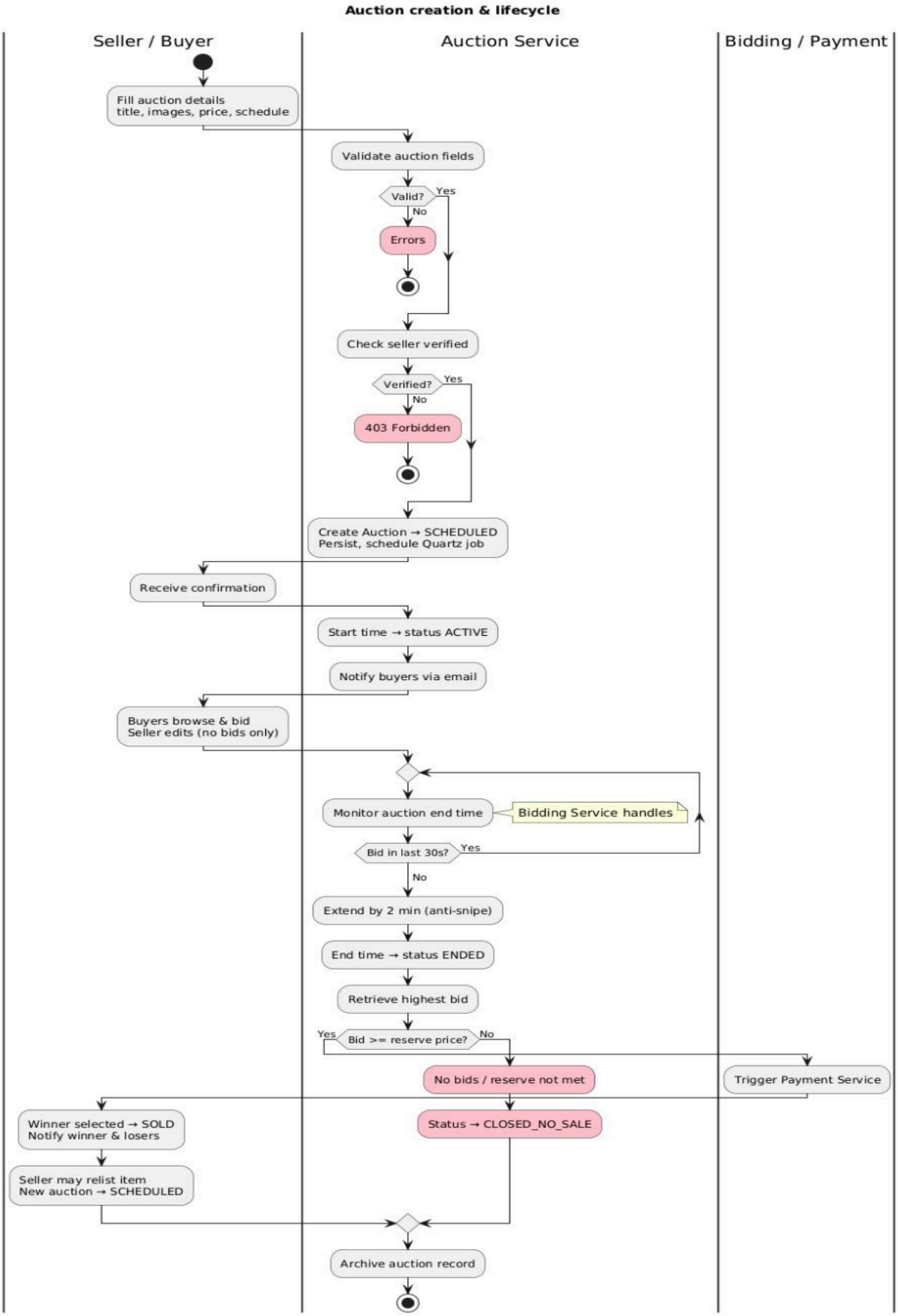
- The Fix in figure 1: Because dependencies are injected via interfaces (like INotificationService), we can easily pass "mock" or "fake" implementations into BidService during testing. We can test our core bidding logic in milliseconds without touching a real database or sending a single email.

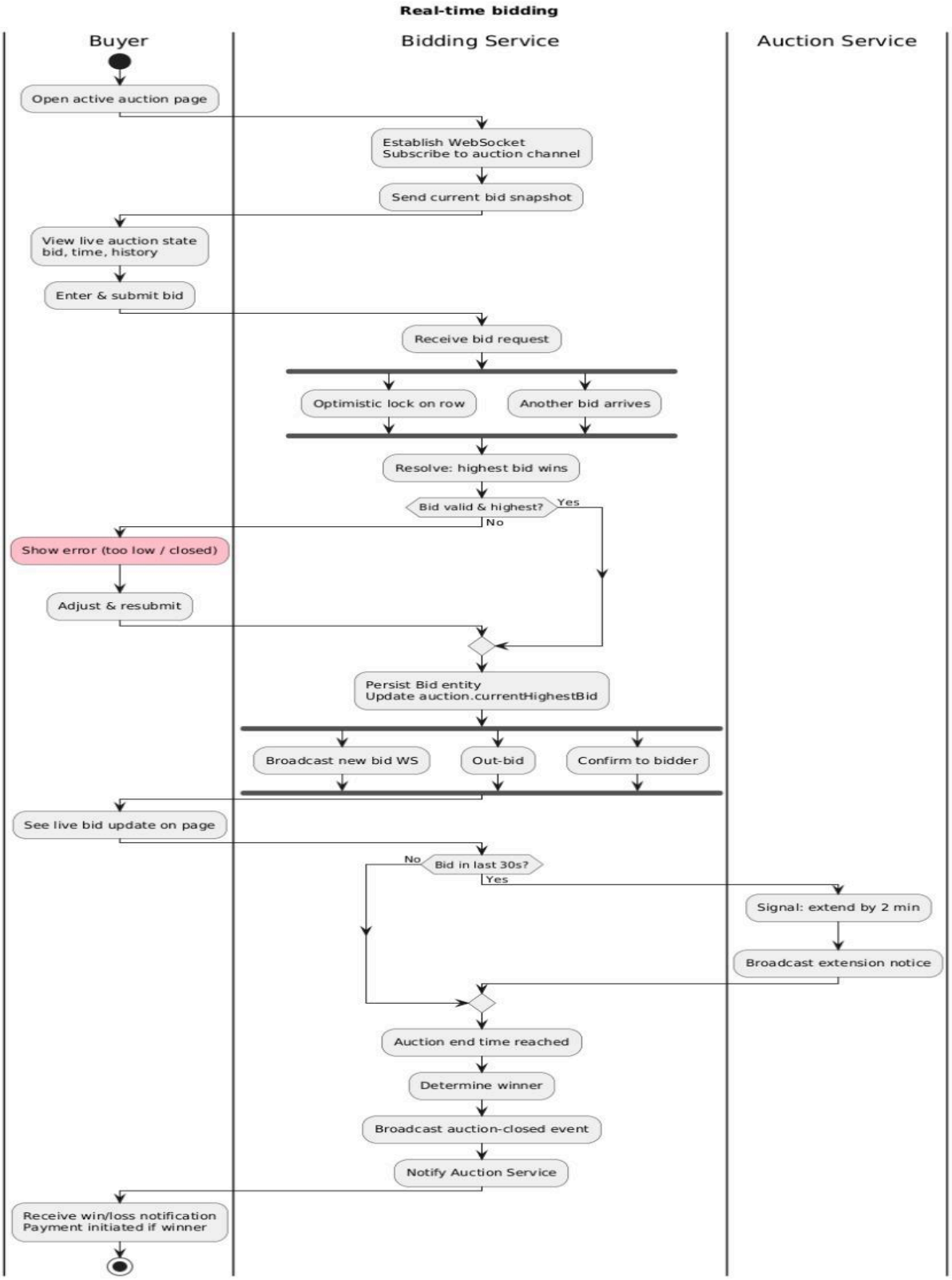
Use Case Diagram

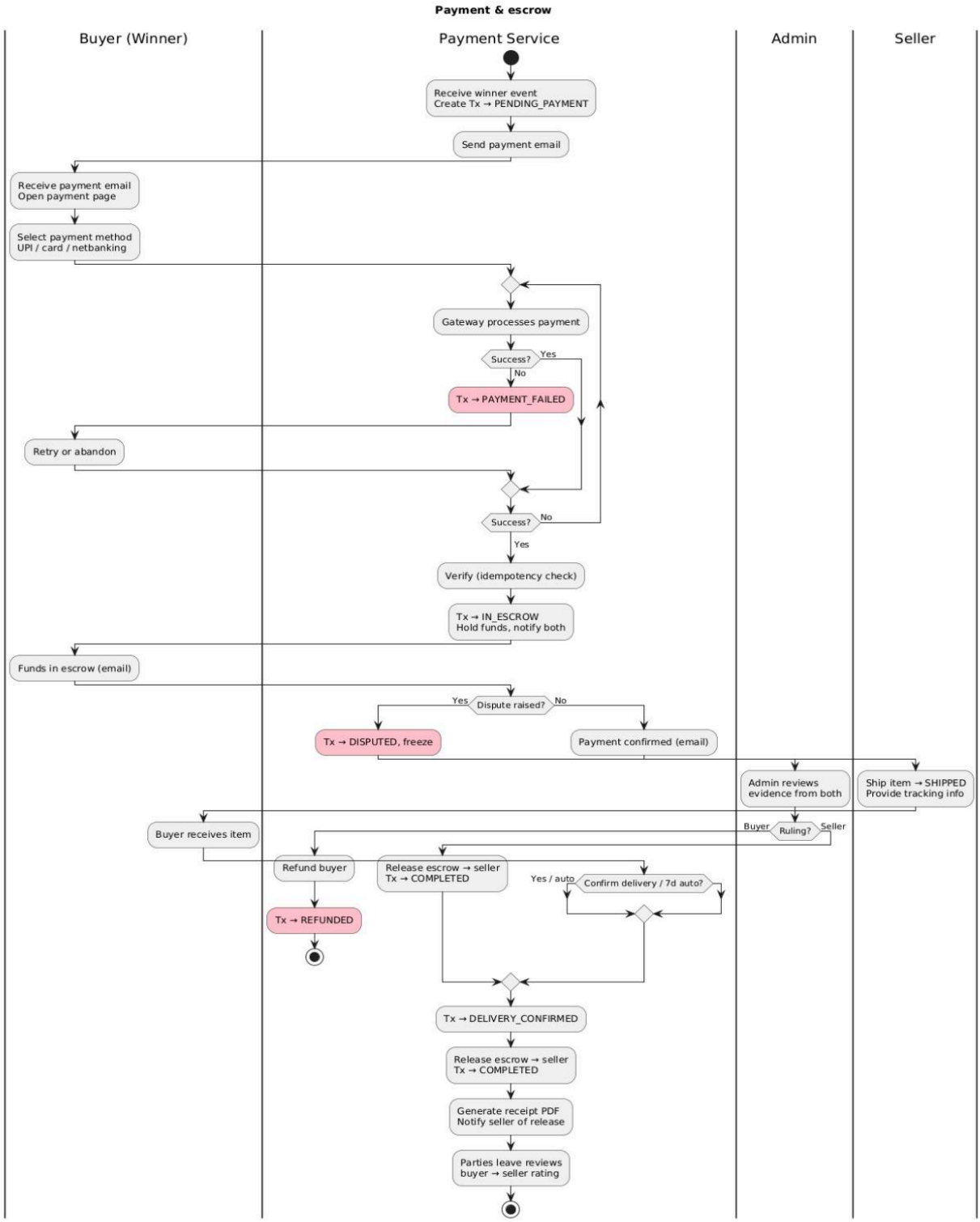


Activity Diagrams

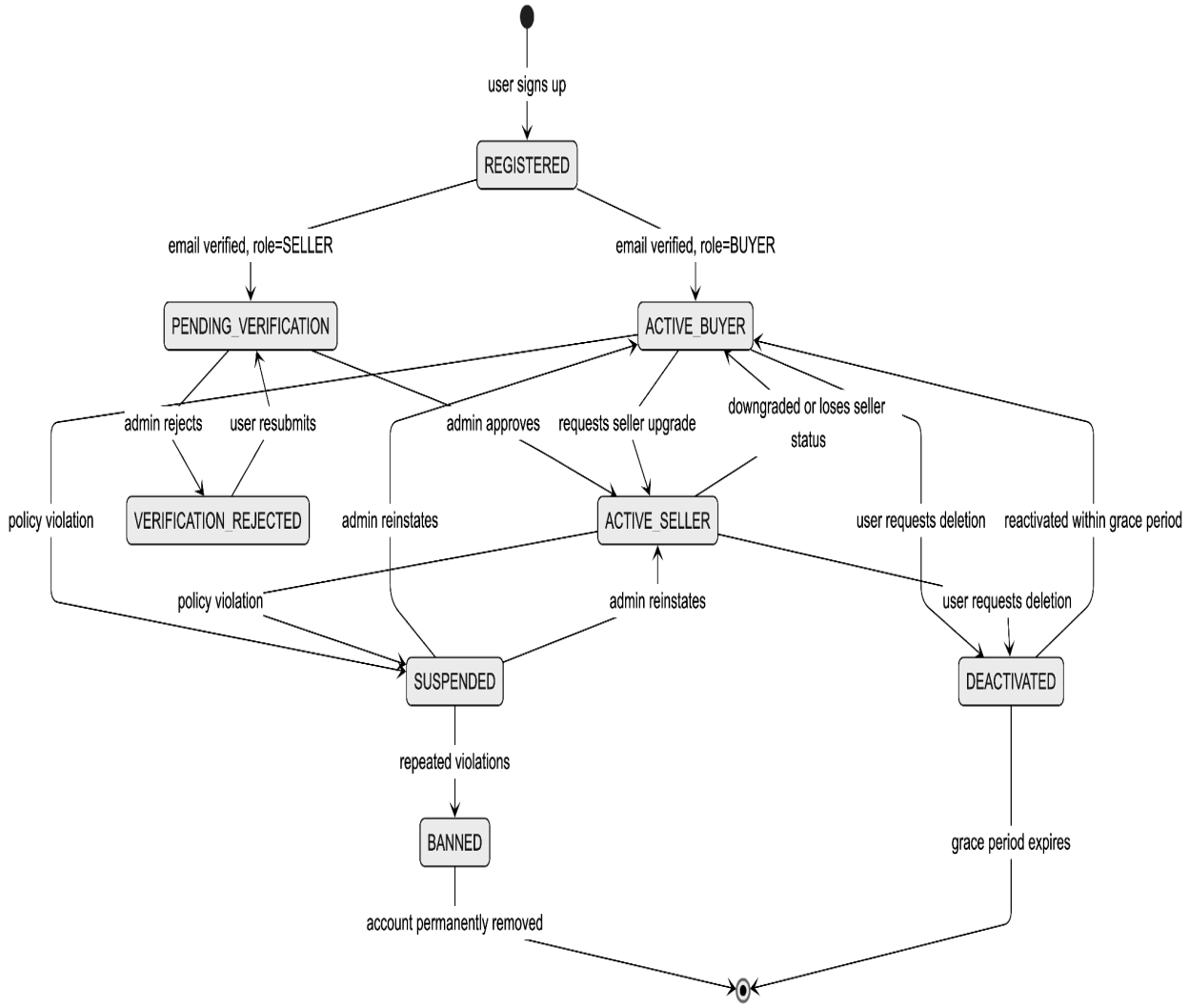


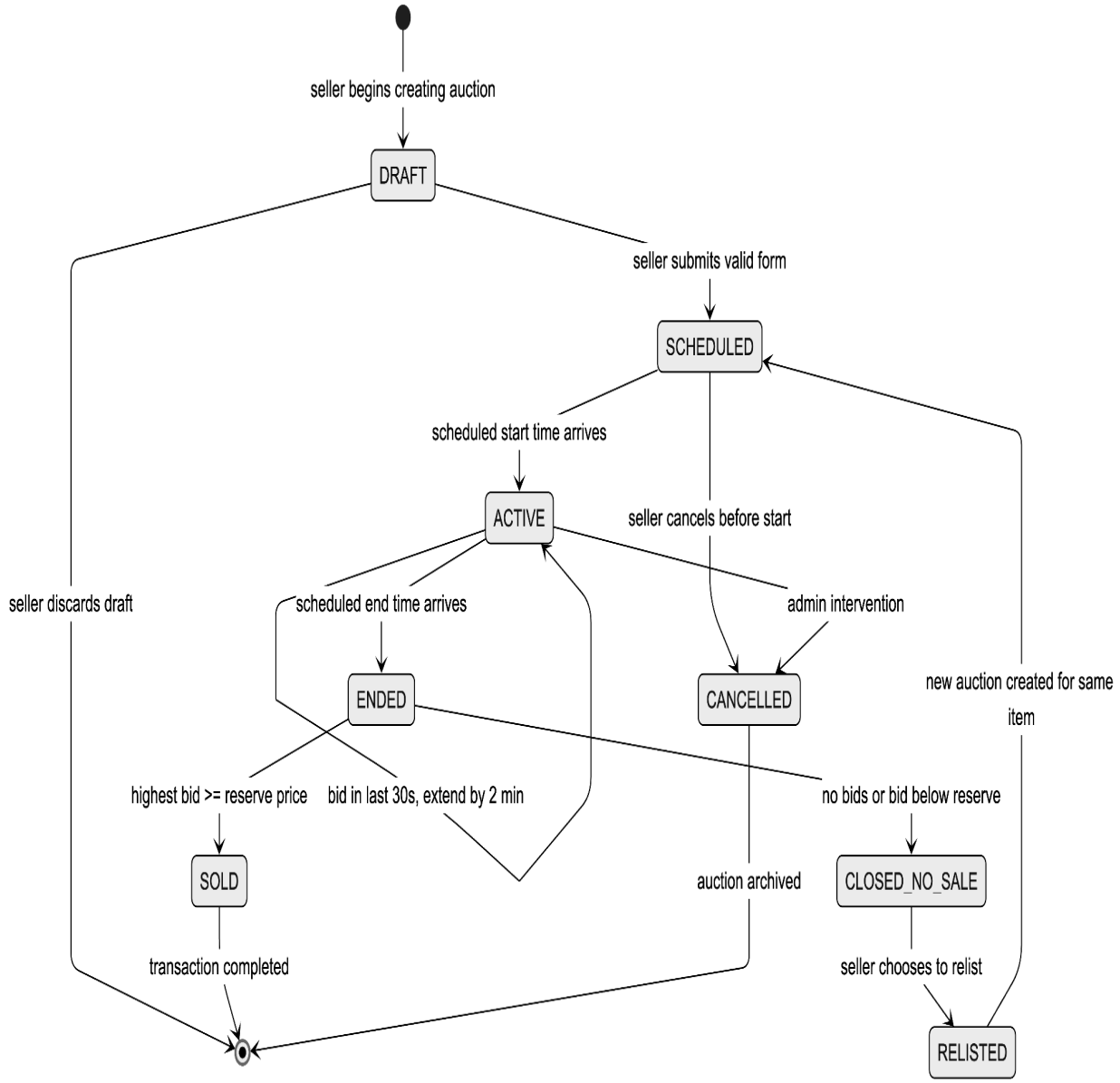

Auction creation & lifecycle
Seller / Buyer
Auction Service
Bidding / Payment
Fill auction details
title, images, price, schedule
Validate auction fields
Valid?
Yes
No
Errors
Check seller verified
Verified?
Yes
No
403 Forbidden
Create Auction → SCHEDULED
Persist, schedule Quartz job
Receive confirmation
Start time → status ACTIVE
Notify buyers via email
Buyers browse & bid
Seller edits (no bids only)
Monitor auction end time
Bidding Service handles
Bid in last 30s?
Yes
No
Extend by 2 min (anti-snipe)
End time → status ENDED
Retrieve highest bid
Bid >= reserve price?
Yes
No
No bids / reserve not met
Trigger Payment Service
Winner selected → SOLD
Notify winner & losers
Status → CLOSED_NO_SALE
Seller may relist item
New auction → SCHEDULED
Archive auction record

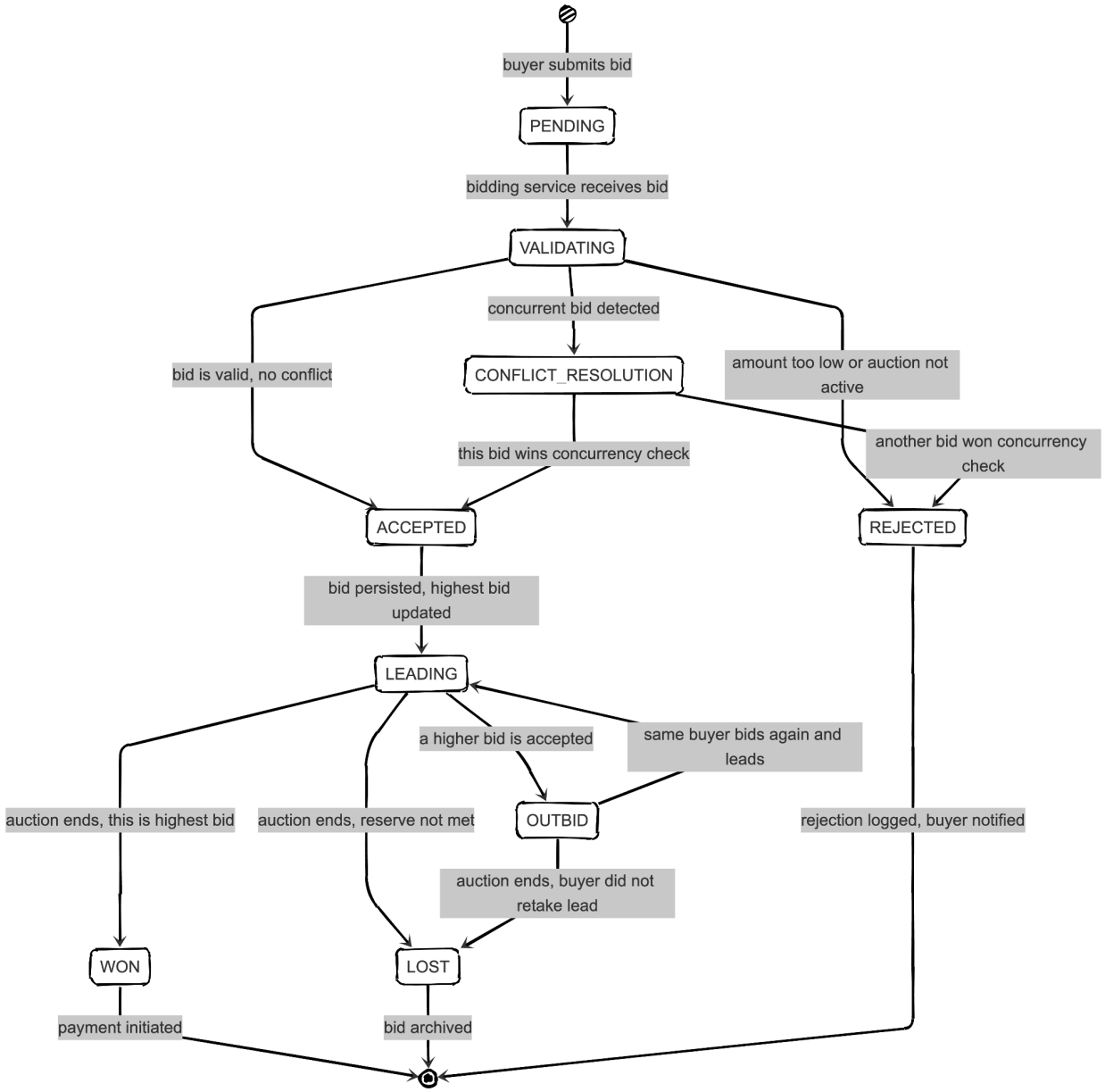

Real-time bidding
Buyer
Bidding Service
Auction Service
Open active auction page
Establish WebSocket
Subscribe to auction channel
Send current bid snapshot
View live auction state
bid, time, history
Enter & submit bid
Receive bid request
Optimistic lock on row
Another bid arrives
Resolve: highest bid wins
Bid valid & highest?
Yes
No
Show error (too low / closed)
Adjust & resubmit
Persist Bid entity
Update auction.currentHighestBid
Broadcast new bid WS
Out-bid
Confirm to bidder
See live bid update on page
Bid in last 30s?
No
Yes
Signal: extend by 2 min
Broadcast extension notice
Auction end time reached
Determine winner
Broadcast auction-closed event
Notify Auction Service
Receive win/loss notification
Payment initiated if winner

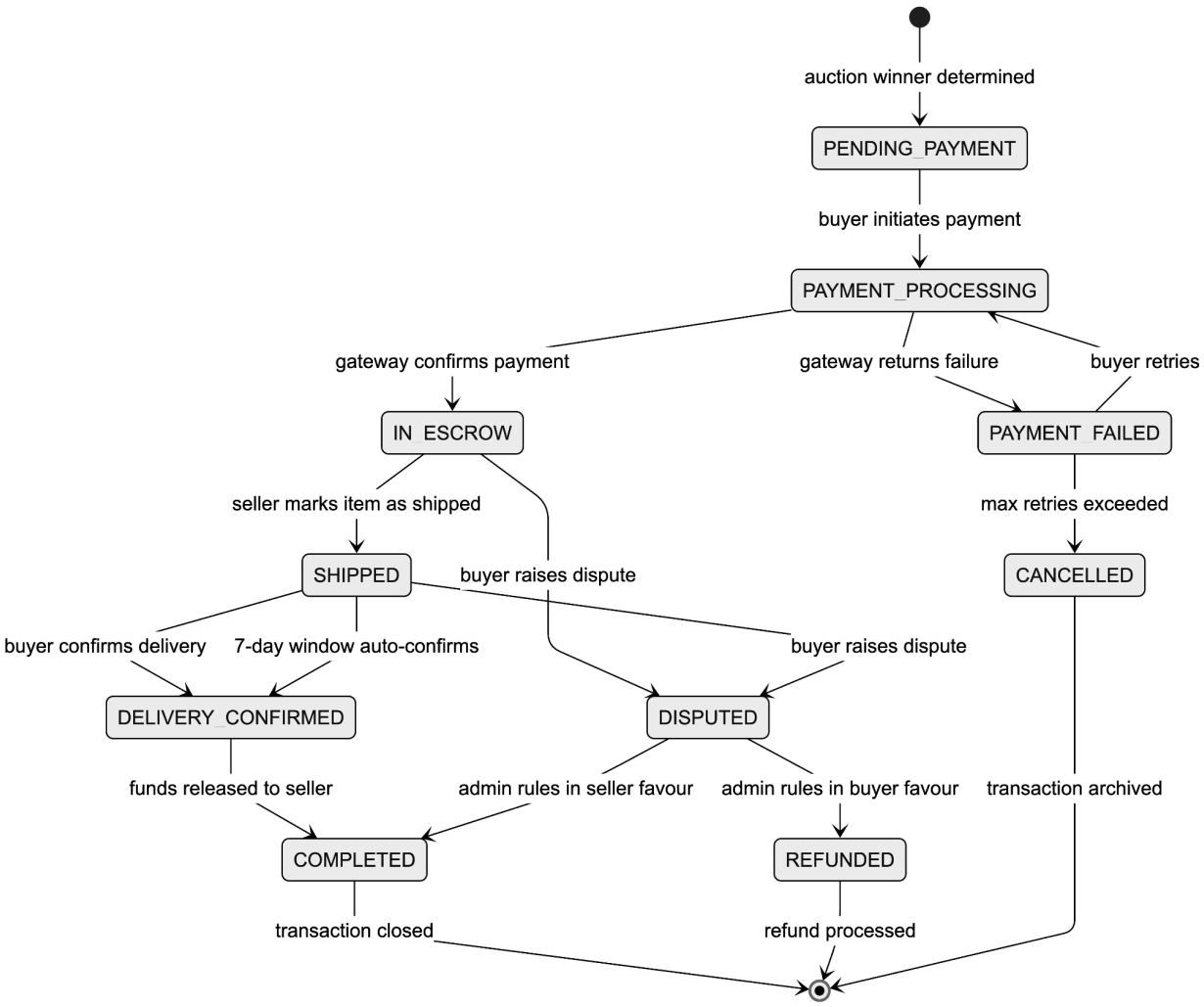

Payment & escrow
Buyer (Winner)
Payment Service
Admin
Seller
Receive winner event
Create Tx → PENDING_PAYMENT
Send payment email
Receive payment email
Open payment page
Select payment method
UPI / card / netbanking
Gateway processes payment
Success?
Yes
No
Tx → PAYMENT_FAILED
Retry or abandon
Success?
No
Yes
Verify (idempotency check)
Tx → IN_ESCROW
Hold funds, notify both
Funds in escrow (email)
Dispute raised?
Yes
No
Tx → DISPUTED, freeze
Payment confirmed (email)
Admin reviews
evidence from both
Ship item → SHIPPED
Provide tracking info
Buyer receives item
Ruling?
Buyer
Seller
Refund buyer
Release escrow → seller
Tx → COMPLETED
Yes / auto
Confirm delivery / 7d auto?
Tx → REFUNDED
Tx → DELIVERY_CONFIRMED
Release escrow → seller
Tx → COMPLETED
Generate receipt PDF
Notify seller of release
Parties leave reviews
buyer → seller rating

State Diagrams




seller begins creating auction
DRAFT
seller submits valid form
SCHEDULED
scheduled start time arrives
ACTIVE
seller cancels before start
seller discards draft
scheduled end time arrives
admin intervention
ENDED
CANCELLED
new auction created for same item
highest bid >= reserve price
bid in last 30s, extend by 2 min
no bids or bid below reserve
SOLD
auction archived
CLOSED_NO_SALE
transaction completed
seller chooses to relist
RELISTED


buyer submits bid
PENDING
bidding service receives bid
VALIDATING
concurrent bid detected
bid is valid, no conflict
CONFLICT_RESOLUTION
amount too low or auction not active
this bid wins concurrency check
another bid won concurrency check
ACCEPTED
REJECTED
bid persisted, highest bid updated
LEADING
a higher bid is accepted
same buyer bids again and leads
auction ends, this is highest bid
auction ends, reserve not met
OUTBID
rejection logged, buyer notified
auction ends, buyer did not retake lead
WON
LOST
payment initiated
bid archived


auction winner determined
PENDING_PAYMENT
buyer initiates payment
PAYMENT_PROCESSING
gateway confirms payment
gateway returns failure
buyer retries
IN_ESCROW
PAYMENT_FAILED
seller marks item as shipped
max retries exceeded
SHIPPED
buyer raises dispute
CANCELLED
buyer confirms delivery
7-day window auto-confirms
buyer raises dispute
DELIVERY_CONFIRMED
DISPUTED
funds released to seller
admin rules in seller favour
admin rules in buyer favour
transaction archived
COMPLETED
REFUNDED
transaction closed
refund processed